

INCIDENT INVESTIGATION REPORT

SSH Brute Force Attack — Zabbix SIEM Home lab

Prepared by: Denin Sajan

Role: Independent Security Researcher

Date: 1 July 2026

Classification: TLP:WHITE (Public — Portfolio Use)

1. Executive Summary

This report documents a simulated SSH brute force attack conducted as part of a cybersecurity home lab project. The goal was to build and validate a working SIEM pipeline capable of detecting credential-based attacks in real time.

The attack was launched from a Kali Linux VM (192.168.64.12) against an Ubuntu 26.04 LTS server (192.168.64.13) running Zabbix 7.4.11. Using Hydra and a 1,000-entry subset of the RockYou (2009) from Daniel Miessler, SecLists, GitHub — <https://github.com/danielmiessler/SecLists> leaked password corpus, repeated failed SSH login attempts were generated against the user account 'denin77'. Zabbix successfully detected the attack, fired a high-severity alert labelled 'SSH Brute Force Detected', and logged the evidence trail — all within minutes of the attack starting.

No systems were compromised. This was a controlled, isolated lab environment with no connection to production infrastructure.

2. Incident Metadata

Incident ID	INC-2026-001
Date & Time	2026-07-01, ~04:03 AM UTC
Detection Method	Zabbix SIEM — UserParameter + trigger (find/like)
Alert Severity	High
Attack Type	SSH Brute Force (Credential Stuffing)
Attacker IP	192.168.64.12 (Kali Linux VM)
Target IP	192.168.64.13 (Ubuntu 26.04 LTS Server)
Target Service	SSH (Port 22)
Target Account	denin77
Tool Used	Hydra v9.6
Wordlist	RockYou 2009 breach — 1,000 entry subset
Status	Resolved — Controlled simulation, no breach

3. Lab Environment

3.1 Infrastructure

- Host machine: MacBook (Apple Silicon, UTM hypervisor)
- SIEM server: Ubuntu 26.04 LTS (ARM64) — IP: 192.168.64.13
- Attacker machine: Kali Linux VM — IP: 192.168.64.12
- Both VMs on the same isolated UTM virtual network

3.2 Stack

- Zabbix Server 7.4.11 — monitoring and alerting
- MySQL 8.4.10 — Zabbix backend database
- Zabbix Agent — installed on the Ubuntu server, polling /var/log/auth.log every 30 seconds
- Apache2 — serving the Zabbix web frontend
- Hydra v9.6 — SSH password attack tool

4. Attack Narrative

4.1 Pre-Attack Setup

Before launching the attack, I verified that Kali could reach the target server over the network:

```
ping 192.168.64.13
```

12 packets transmitted, 0% packet loss, average RTT ~1.3ms — connectivity confirmed.

A 1,000-entry wordlist was extracted from the RockYou dataset already present on Kali at /usr/share/wordlists/rockyou.txt:

```
head -n 1000 /usr/share/wordlists/rockyou.txt > /tmp/passlist.txt
```

4.2 Brute Force Execution

Hydra was launched with 2 parallel threads and a 3-second wait between retries to avoid triggering SSH's MaxStartups connection limit (which we had hit in an earlier test run):

```
hydra -l denin77 -P /tmp/passlist.txt ssh://192.168.64.13 -t 2 -W 3
```

Hydra attempted each of the 1,000 passwords sequentially against the 'denin77' account. Each failed attempt generated multiple log entries in /var/log/auth.log on the target server, including:

```
Failed password for denin77 from 192.168.64.12 port XXXXX ssh2
password check failed for user (denin77)
pam_unix(sshd:auth): authentication failure
```

SSH's built-in protection also triggered mid-run, disconnecting sessions that exceeded 6 consecutive failures with the message 'Too many authentication failures'. This is normal sshd behaviour and was captured in the logs as evidence of defensive controls operating alongside our detection.

4.3 Timeline

~03:01 UTC	First Hydra run attempted (4 threads) — hit MaxStartups limit, connection errors
~03:13 UTC	Second Hydra run — sudo permissions fixed for Zabbix agent, auth.log access confirmed
~04:03 UTC	Third Hydra run launched (2 threads, -W 3)
04:03:09 UTC	Zabbix trigger fired — 'SSH Brute Force Detected' alert raised (High severity)
04:07 UTC	Alert confirmed in Monitoring → Problems dashboard, duration 3m 57s

5. Detection Analysis

5.1 How Detection Was Configured

The detection pipeline was built entirely within Zabbix using three components:

UserParameter in zabbix_agentd.conf — this tells the Zabbix agent to run a shell command when polled:

```
UserParameter=ssh.auth.log.sudo /usr/bin/tail -n 100 /var/log/auth.log
```

Every 30 seconds, the agent reads the last 100 lines of the SSH auth log and sends the output to the Zabbix server as a text value.

Zabbix Item — a monitoring item named 'SSH Failed Logins' (type: Zabbix agent, key: ssh.auth.log, data type: Text) — was created on the Zabbix server host to receive and store the log data.

Zabbix Trigger — a high-severity trigger named 'SSH Brute Force Detected' fires when the collected text contains the string 'Failed password':

```
find(/Zabbix server/ssh.auth.log, "like", "Failed password")=1
```

This expression evaluates to 1 (true) any time the most recently collected log snapshot contains that string, immediately raising an alert visible in Monitoring → Problems.

5.2 Evidence Captured

The Zabbix History view for the 'SSH Failed Logins' item showed the raw auth.log content being captured in real time. A sample of the evidence:

```
2026-07-01T03:23:11 siem sshd-session: Failed password for denin77 from 192.168.64.12 port 58410 ssh2
2026-07-01T03:23:13 siem sshd-session: Failed password for denin77 from 192.168.64.12 port 58404 ssh2
2026-07-01T03:23:13 siem sshd-session: error: maximum authentication attempts exceeded for denin77
2026-07-01T03:23:13 siem sshd-session: Disconnecting: Too many authentication failures [preauth]
```

Key IOCs (Indicators of Compromise) identified from the logs:

- Source IP: 192.168.64.12 — consistent across all attempts, single attacker
- Target account: denin77 — all attempts targeting the same username (classic credential-stuffing pattern)
- Port variation: multiple source ports used (58410, 58404, 48674, 41918...) — typical of Hydra's parallel connections
- Frequency: multiple failed attempts per second sustained over several minutes
- Pattern: repeated 'maximum authentication attempts exceeded' followed by forced disconnection, then reconnection — clear automated tool signature

6. Findings

6.1 What Worked

- The SIEM pipeline detected the brute force attack successfully and raised a High-severity alert within seconds of the 'Failed password' string appearing in the log
- The Zabbix trigger fired at 04:03:09 AM — well within the 30-second polling interval
- SSH's built-in MaxStartups and MaxAuthTries controls disrupted the attack mid-run, demonstrating layered defence
- The auth.log provided a rich, timestamped evidence trail sufficient for a full investigation

6.2 Gaps Identified

- The trigger fires on a single occurrence of 'Failed password' — in production, this generates false positives from legitimate failed logins. A threshold-based trigger (e.g. more than 10 failures in 60 seconds) would be more appropriate
- No automated response was configured — a real SOC would pair this with an action to block the source IP via firewall rule or notify an analyst via email/Slack
- The wordlist used is common and would be exhausted quickly by a real attacker who would pivot to more targeted lists
- No MFA was configured on SSH — public key authentication or 2FA would have stopped this attack entirely

7. Recommendations

- Disable password authentication for SSH and enforce key-based authentication only (PasswordAuthentication no in sshd_config)
- Install and configure fail2ban to automatically ban IPs after a configurable number of failed attempts
- Refine the Zabbix trigger to use a count-based threshold rather than single-occurrence detection

- Configure Zabbix action to send an email or webhook alert when the trigger fires
- Enable SSH rate limiting at the firewall level
- Implement centralised log aggregation so auth.log data feeds into a full SIEM pipeline rather than a 100-line tail

8. Tools & References

Zabbix 7.4.11	https://www.zabbix.com
Hydra v9.6	https://github.com/vanhauser-thc/thc-hydra
RockYou dataset	Daniel Miessler, SecLists, GitHub- https://github.com/danielmiessler/SecLists
MITRE ATT&CK	T1110.001 — Brute Force: Password Guessing
Ubuntu 26.04 LTS	https://ubuntu.com
GitHub Repo	https://github.com/Denin007/denin-sajan

9. Conclusion

This lab gave me hands-on experience with the full incident detection lifecycle — from infrastructure setup and SIEM configuration through to attack simulation, alert validation, and post-incident analysis. The most valuable part wasn't just getting the alert to fire; it was understanding why each component of the pipeline matters and where the gaps are.

Building this entirely from scratch — including troubleshooting a broken MySQL install, dpkg lock conflicts, debconf issues, and Zabbix agent permission problems — gave me a realistic picture of what SOC infrastructure looks like when it doesn't cooperate, which is most of the time.

The project demonstrates practical competency in SIEM deployment and configuration, log analysis, attack pattern recognition, UserParameter scripting, trigger expression writing, and incident documentation.